Sorting Techniques

Shivank Chauhan
Assistant Professor
CEA Department
GLA University, Mathura

Sorting

- Sorting refers to arranging data in a particular order (ascending or descending order).

Classification of Sorting Algorithms

- Internal Sorting
- External Sorting

Internal sorting:

- All the data to sort is stored in memory at all times while sorting is in progress.
  - **Bubble Sort**
  - **Insertion Sort**
  - **Selection sort**
  - Quick Sort
  - Heap Sort
  - Radix Sort

External sorting

- Data is stored outside memory (like on disk) and only loaded into memory in small chunks.
- External sorting is usually applied in cases when data can't fit into memory entirely.
- The external sorting algorithm has to deal with loading and unloading chunks of data in optimal manner.
  - Merge Sort

Bubble Sort

- Simple sorting algorithm.
- Bubble sort will start by comparing the first element of the array with the second element.
- If the first element is greater than the second element.

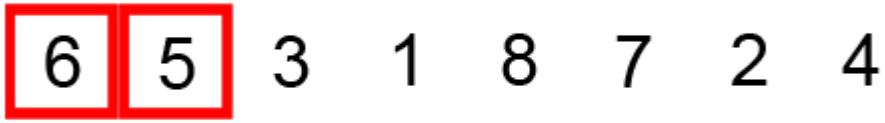
- It will swap both the elements.
- and then move on to compare the second and the third element, and so on.
- Not suitable for large data sets.

A unsorted array of 8 elements

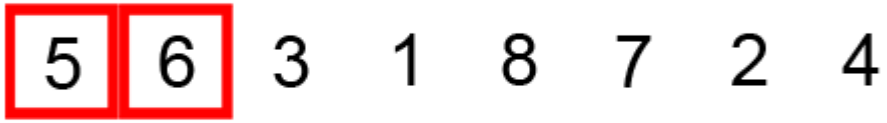
6 5 3 1 8 7 2 4

Pass 1

Comparison: 1
Swapping

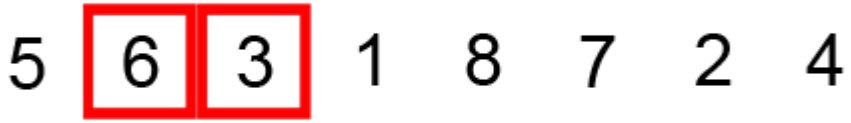


Pass 1

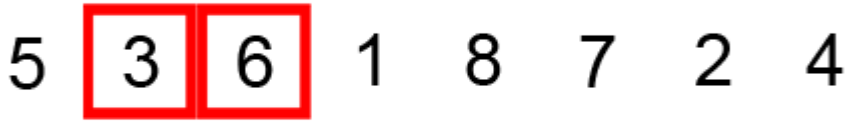


Pass 1

Comparison: 2
Swapping



Pass 1



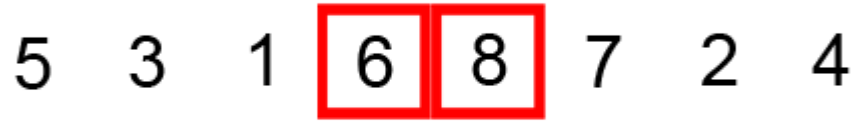
Pass 1

Comparison: 3
Swapping

Pass 1

Pass 1

Comparison: 4
No Swapping



Pass 1

Comparison: 5
Swapping

Pass 1

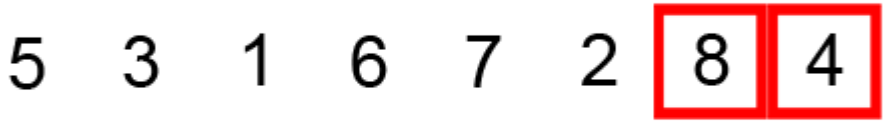
Pass 1

Comparison: 6
Swapping

Pass 1

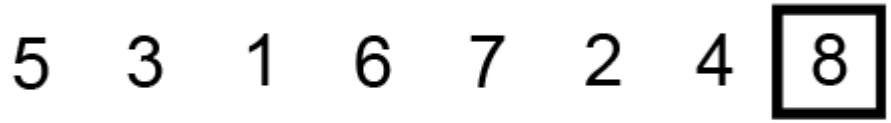
Pass 1

Comparison: 7
Swapping



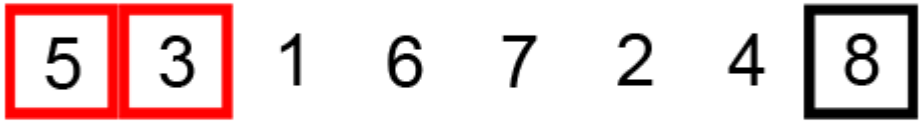
Pass 1

Pass 1 Completed

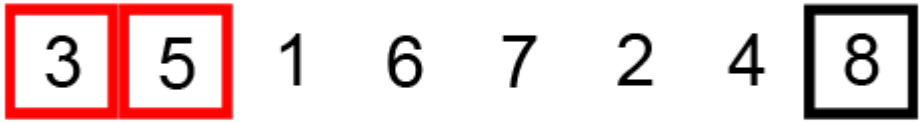


Pass 2

Comparison: 1
Swapping

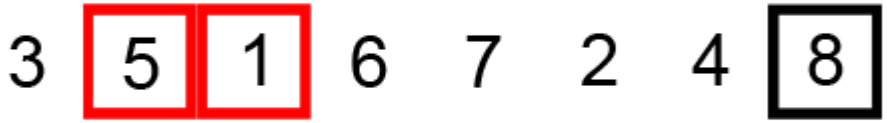


Pass 2

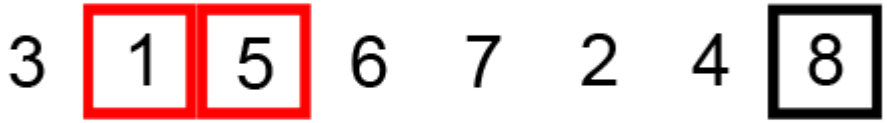


Pass 2

Comparison: 2
Swapping



Pass 2



Pass 2

Comparison: 3
No Swapping

Pass 2

Comparison: 4
No Swapping

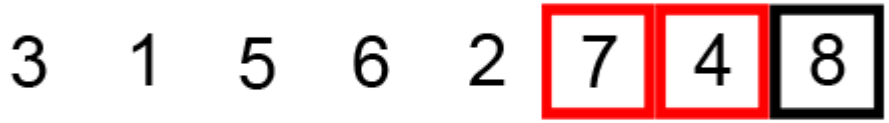
Pass 2

Comparison: 5
Swapping

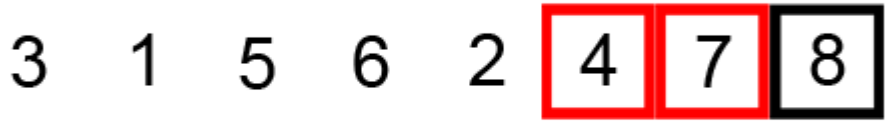
Pass 2

Pass 2

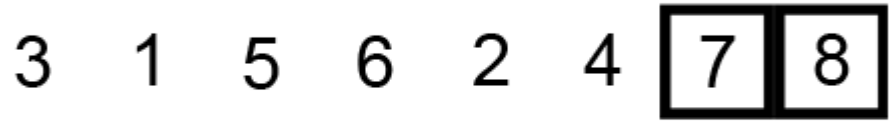
Comparison: 6
Swapping



Pass 2

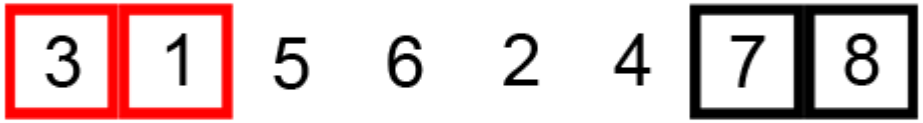


Pass 2 Completed

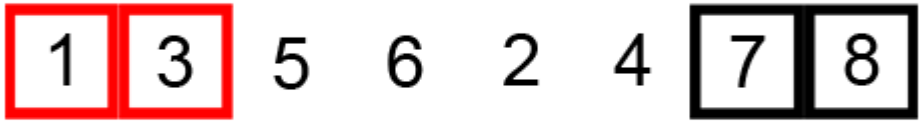


Pass 3

Comparison: 1
Swapping

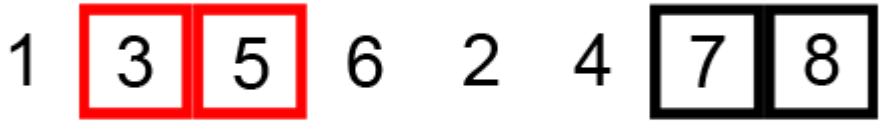


Pass 3



Pass 3

Comparison: 2
No Swapping



Pass 3

Comparison: 3
No Swapping

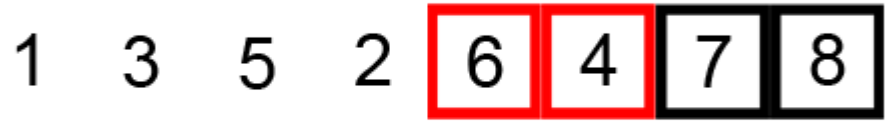
Pass 3

Comparison: 4
Swapping

Pass 3

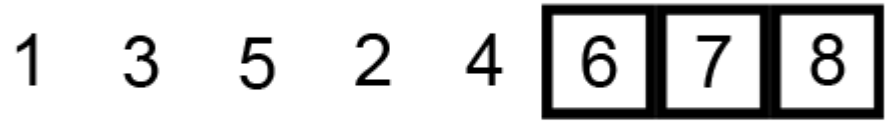
Pass 3

Comparison: 5
Swapping



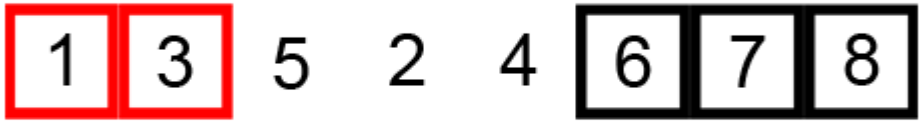
Pass 3

Pass 3 Completed



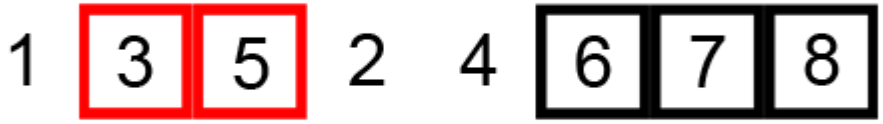
Pass 4

Comparison: 1
No Swapping



Pass 4

Comparison: 2
No Swapping



Pass 4

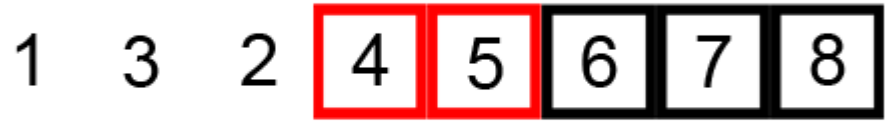
Comparison: 3
Swapping

Pass 4

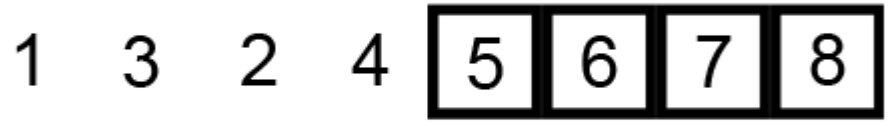
Pass 4

Comparison: 4
Swapping

Pass 4

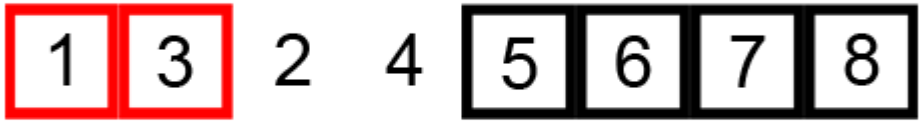


Pass 4 Completed



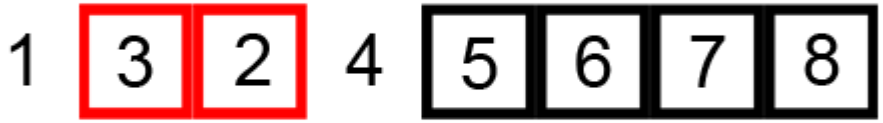
Pass 5

Comparison: 1
No Swapping

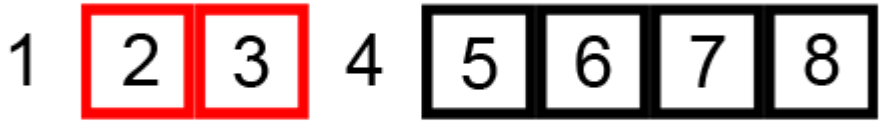


Pass 5

Comparison: 2
Swapping



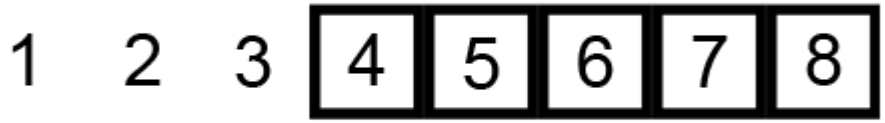
Pass 5



Pass 5

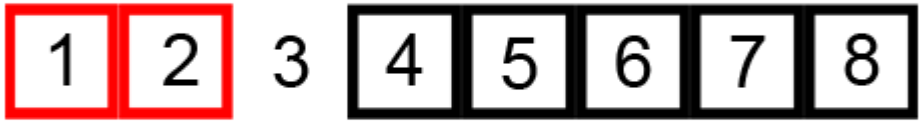
Comparison: 3
No Swapping

Pass 5 Completed



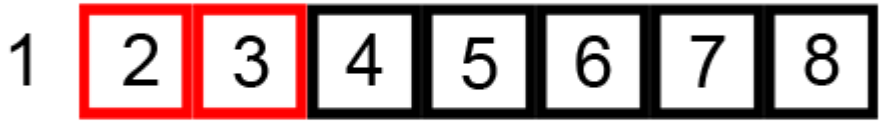
Pass 6

Comparison: 1
No Swapping

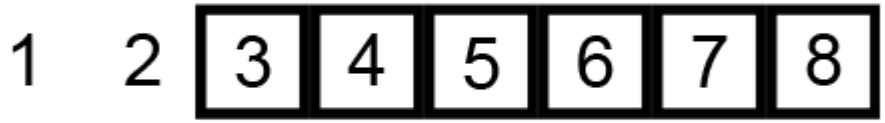


Pass 6

Comparison: 2
No Swapping

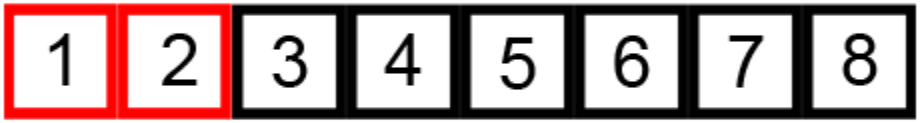


Pass 6 Completed

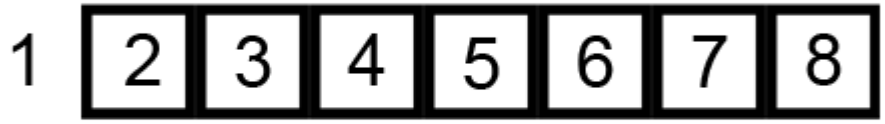


Pass 7

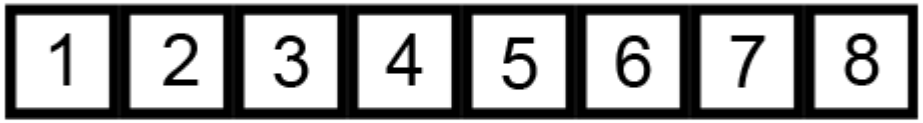
Comparison: 1
No Swapping



Pass 7 Completed



Sorted Array



Bubble Sort Algorithm

Bubblesort (A[n])

Here A is an array of n size and this algorithm sort an array A in ascending order.

Step 1: Start

```
Step 2: for (i = 0; i < ( n - 1 ); i++)
        {
            for (j = 0 ; j < n - i - 1; j++)
             {
                    if (A [j] > A[j+1])
                     {
                          /* Swapping */
                          t  = A[j];
                          A[j]   =  A[j+1];
                          A[j+1] = t;
                      }
                }
        }

Step 3: Stop
```

Complexity Analysis of Bubble Sort

- In Bubble Sort, n-1 comparisons will be done in the 1st pass, n-2 in 2nd pass, n-3 in 3rd pass and so on. So the total number of comparisons will be,

  Sum = (n-1) + (n-2) + (n-3) + ... + 3 + 2 + 1

  Sum = n(n-1)/2 = (n^2-n)/2

  i.e O(n^2)

- Hence the complexity (time complexity) of Bubble Sort is O(n^2).

Insertion Sort

- In-place comparison-based sorting algorithm.
- Here, a sub-list is maintained which is always sorted.
- For example, the lower (left) part of an array is maintained to be sorted.
- An element which is to be inserted in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence the name, insertion sort.

Unsorted Array

6 5 3 1 8 7 2 4

Pass 1

Pass 1

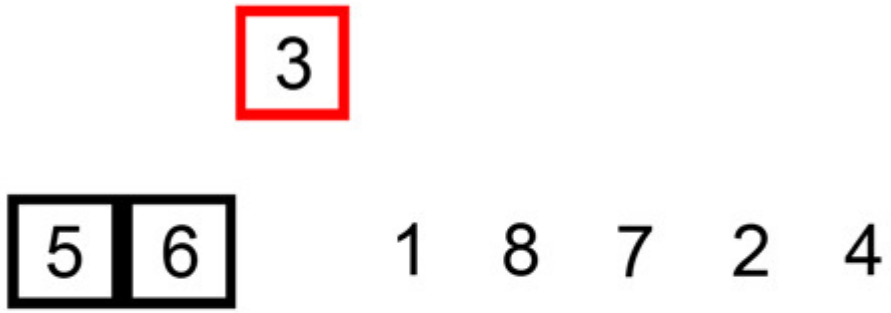
Pass 1

Pass 1

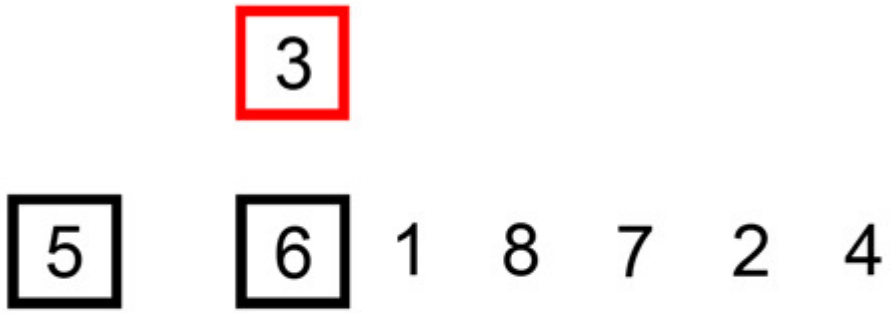
Pass 1 Completed

Pass 2

Pass 2



Pass 2



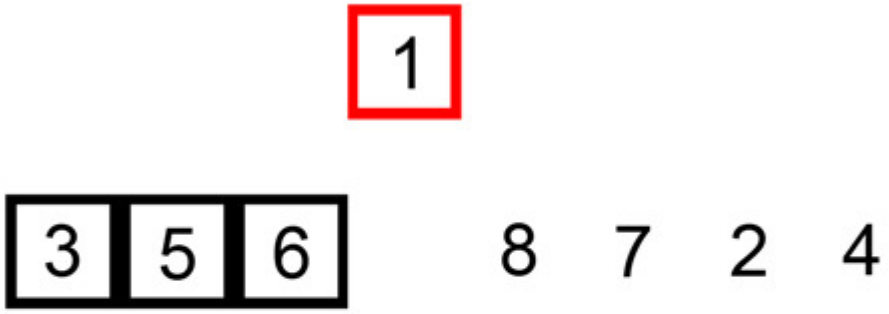
Pass 2

Pass 2

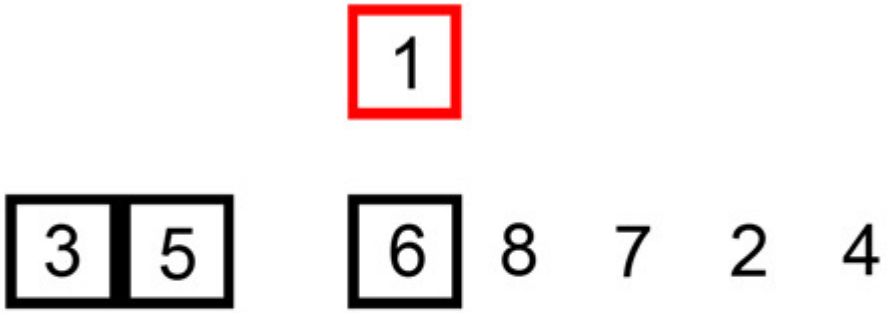
Pass 2 Completed

Pass 3

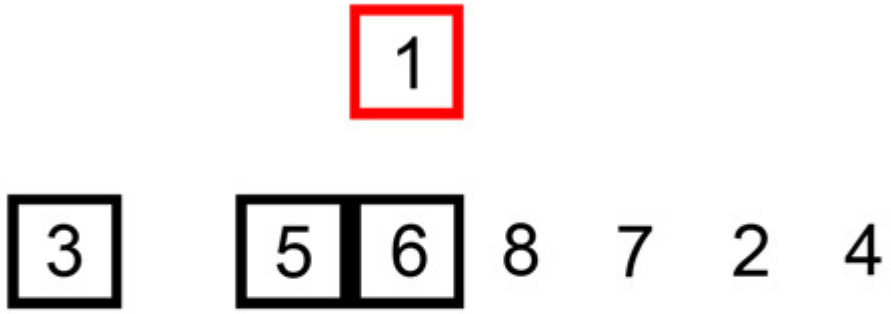
Pass 3



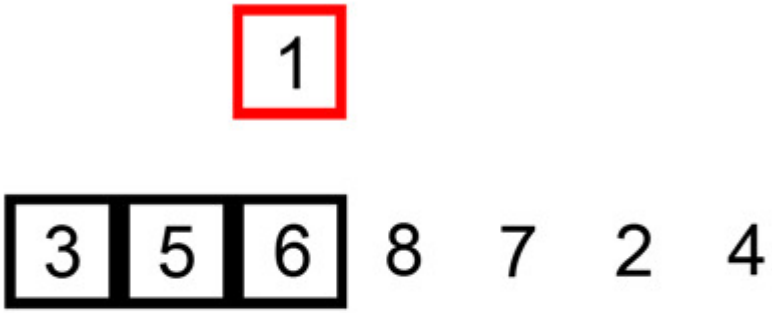
Pass 3



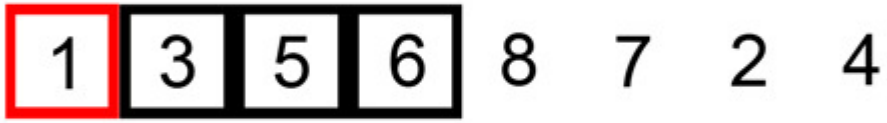
Pass 3



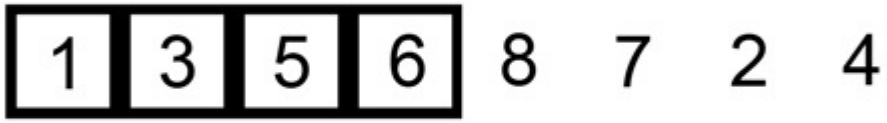
Pass 3



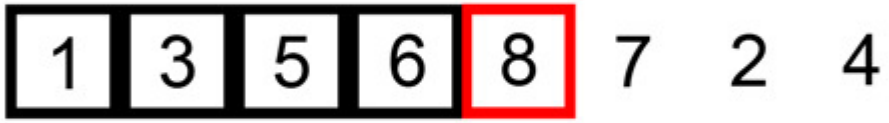
Pass 3



Pass 3 Completed



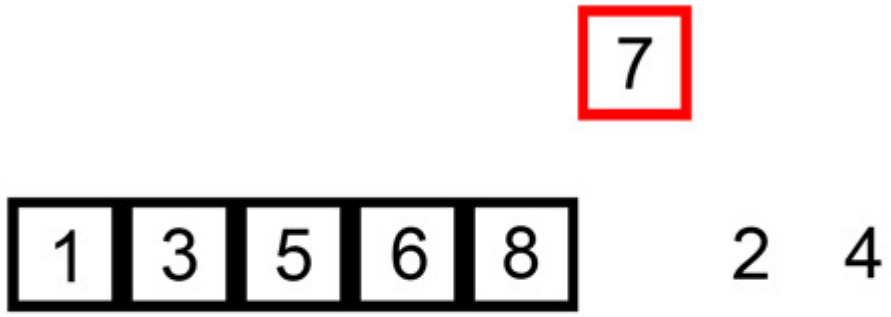
Pass 4



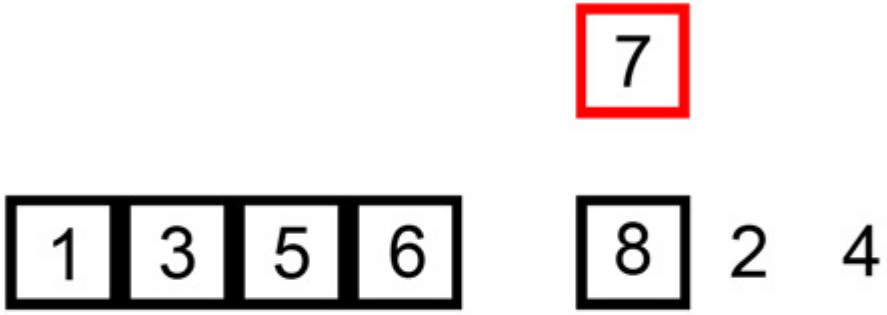
Pass 4 Completed

Pass 5

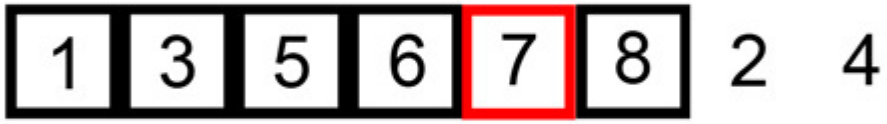
Pass 5



Pass 5

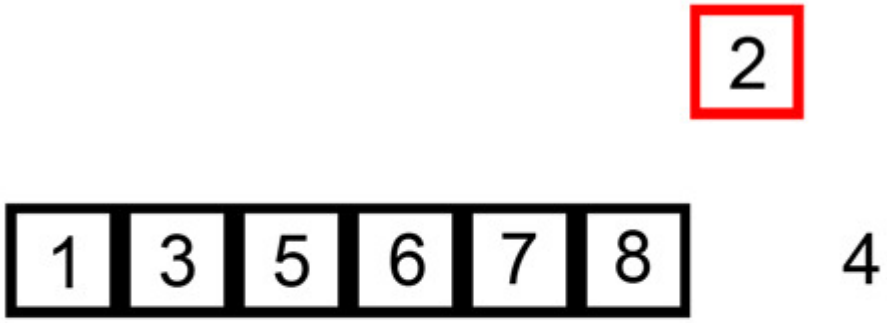


Pass 5

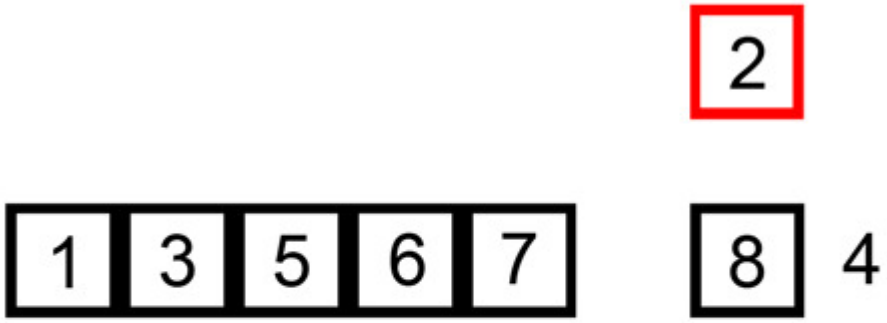


Pass 5 Completed

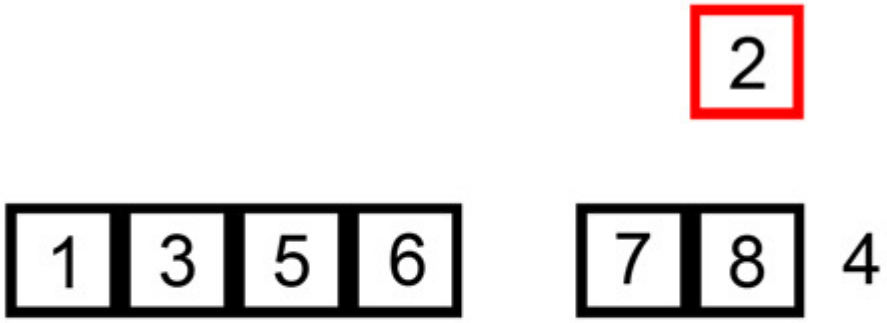
Pass 6



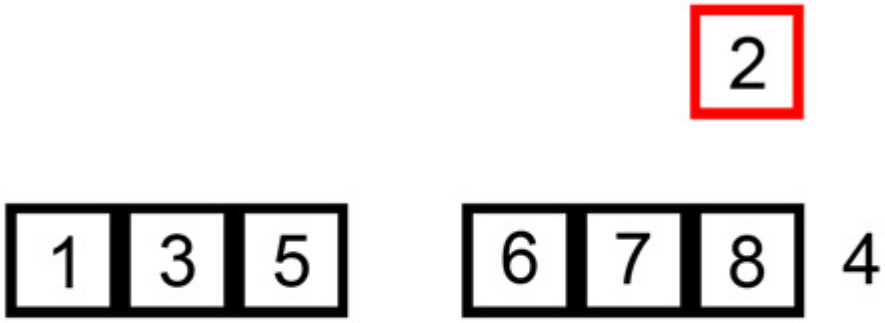
Pass 6



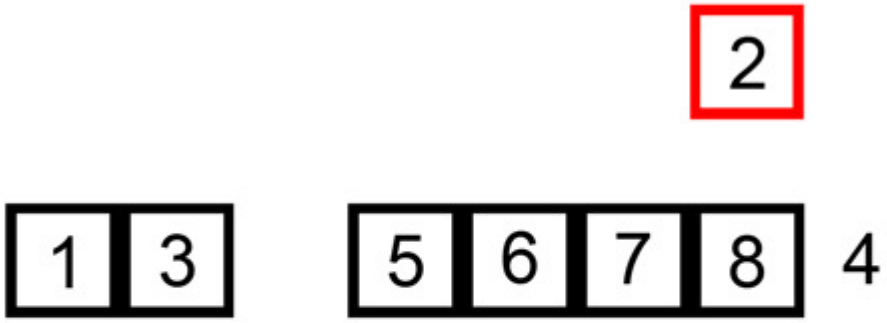
Pass 6



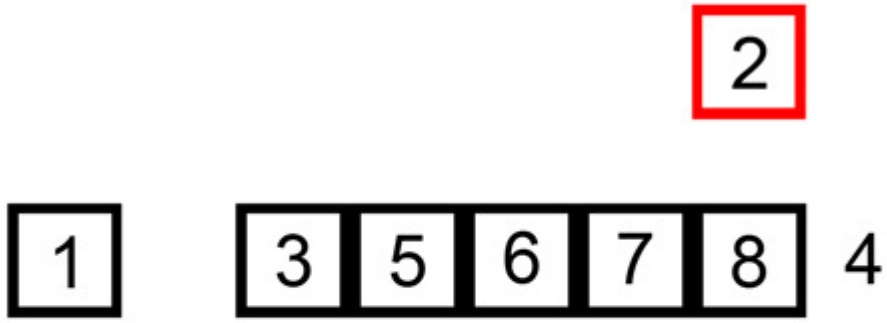
Pass 6



Pass 6



Pass 6

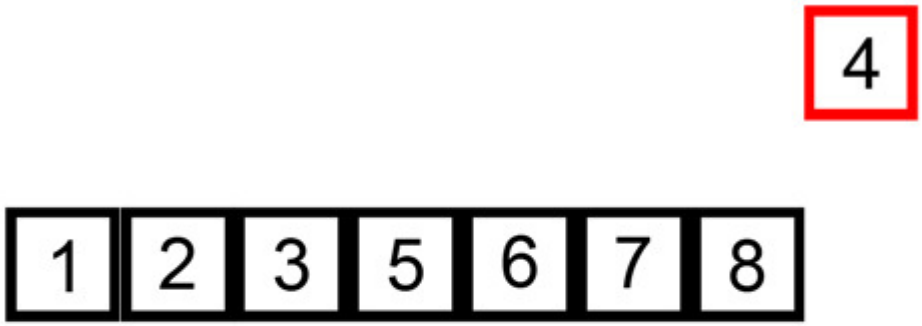


Pass 6

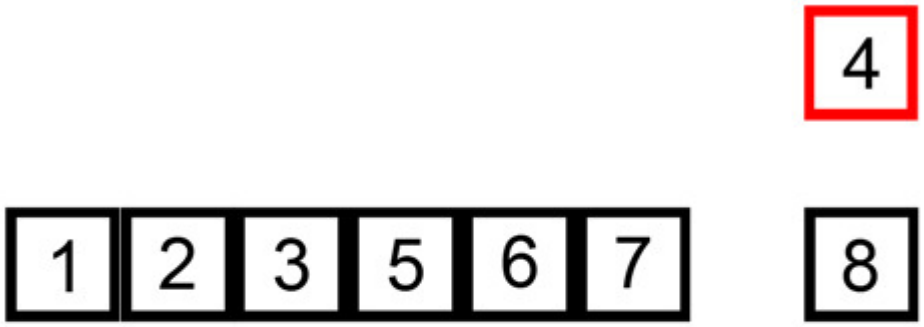
Pass 6 Completed

Pass 7

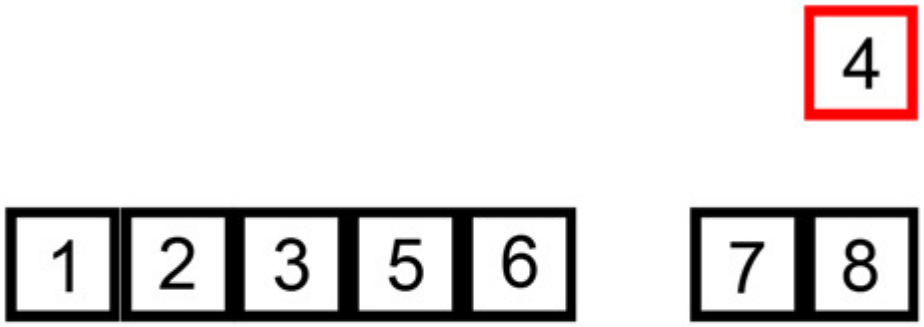
Pass 7



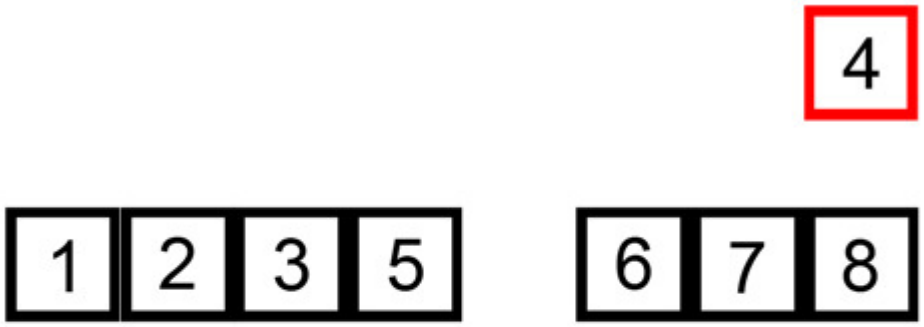
Pass 7



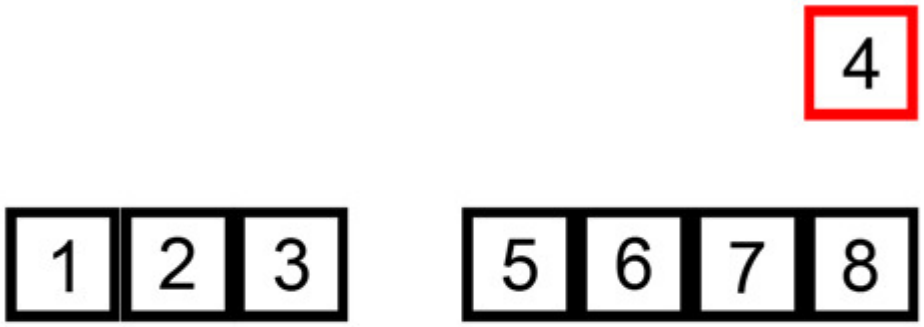
Pass 7



Pass 7



Pass 7



Pass 7

Pass 7 Completed & Sorted Array

Insertion Sort Algorithm

Insertionsort (a[n])

Here a is an array of size n and this algorithm sort an array a in ascending order.

Step 1: Start

Step 2:

```
for (i=1; i<= n -1;i++)
    {
          temp = a[i];
          j=i-1;
          while((temp < a[j] )&&(j>=0))
              {
                    a[j+1] = a[j];
                     j= j-1;
               }
           a[j+1] = temp;
      }
```

Step 3: Stop

Complexity Analysis of Insertion Sort

- In Insertion Sort, 1 comparisons will be done in the 1st pass, 2 in 2nd pass, 3 in 3rd pass and so on. So the total number of comparisons will be,

  Sum = 1 + 2 + 3 + ...+ (n-2) +(n-1)

  Sum = n(n-1)/2 = (n^2-n)/2

  i.e O(n^2)

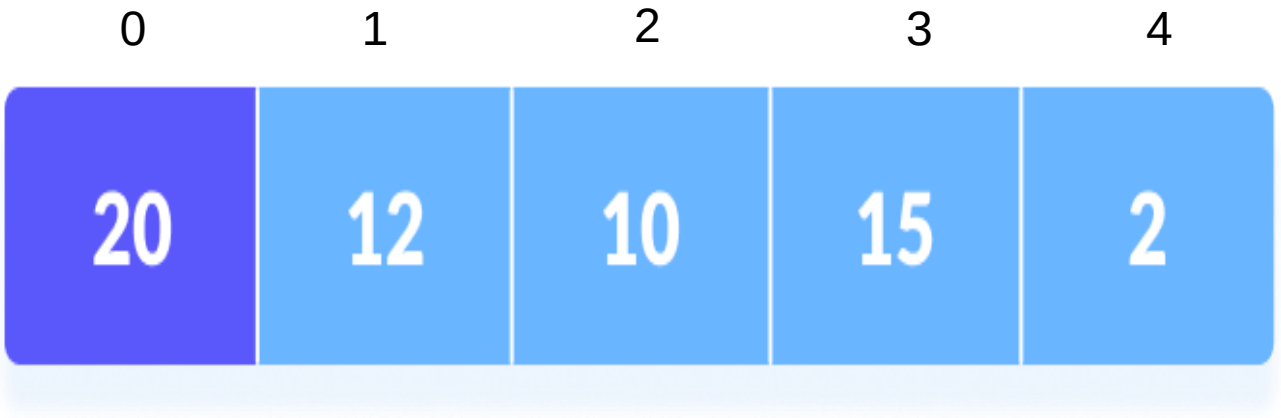
- Hence the complexity (time complexity) of Insertion Sort is O(n^2).

Selection Sort

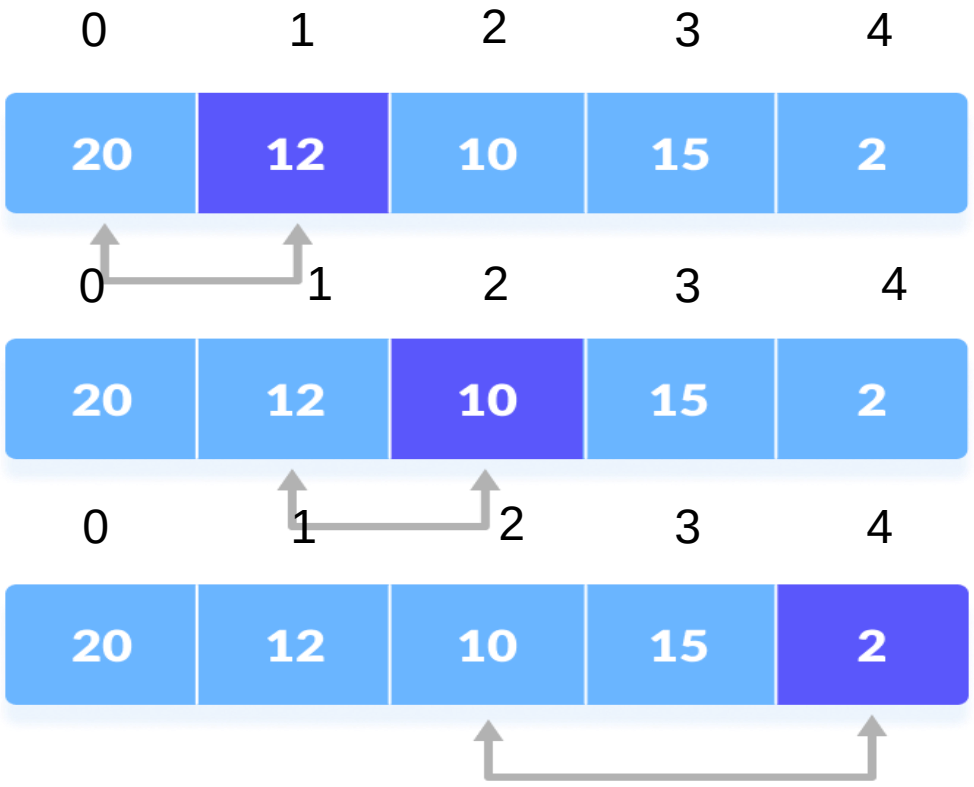
- The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning.

How Selection Sort Works?

1. Set the first element as minimum.

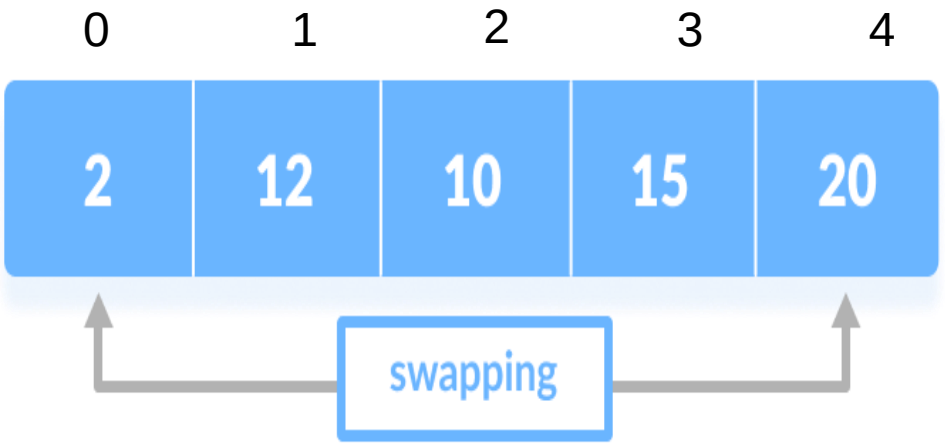


2. Compare minimum with the second element. If the second element is smaller than minimum, assign second element as minimum.



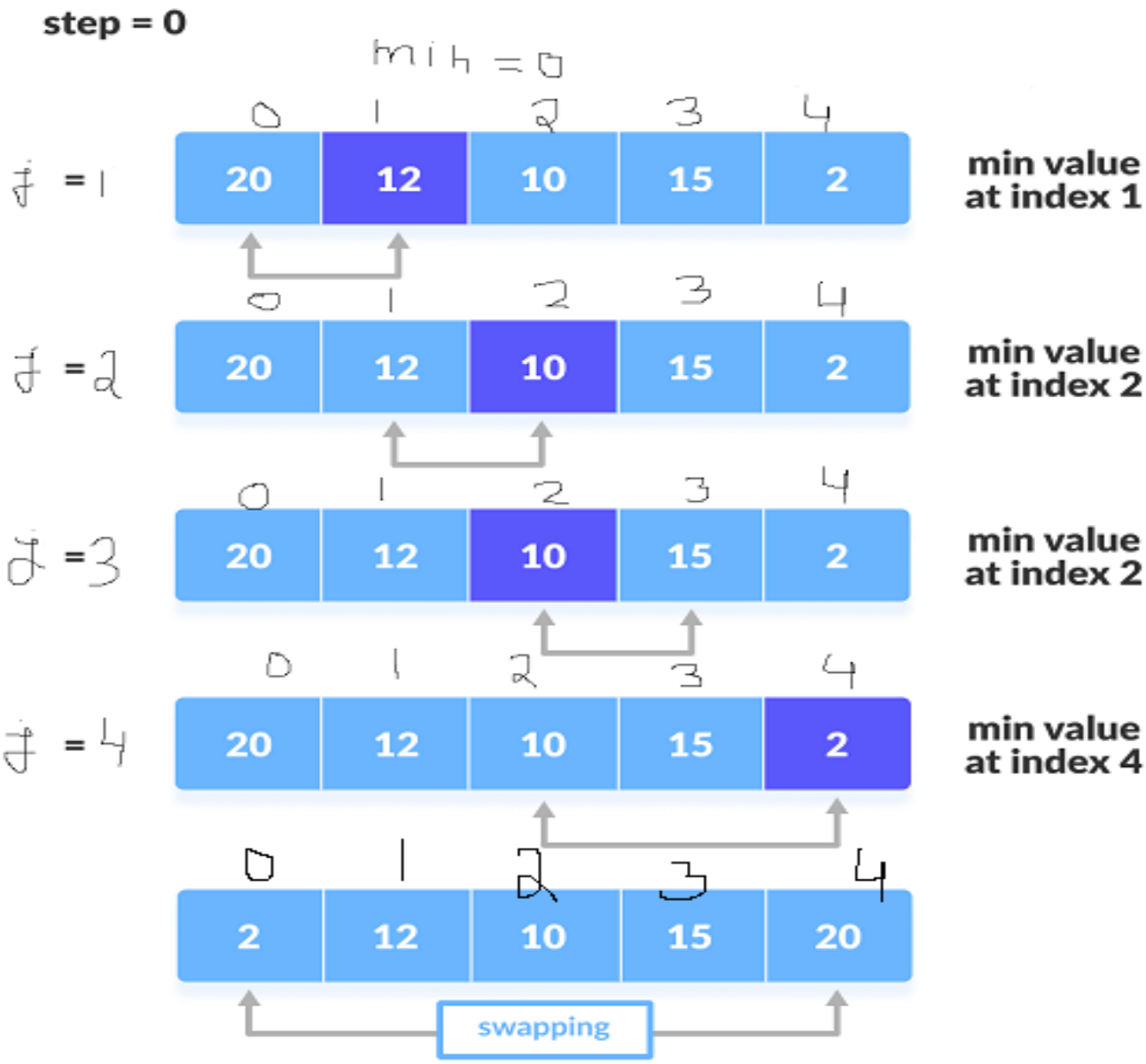
3. Compare minimum with the third element. Again, if the third element is smaller, then assign minimum to the third element otherwise do nothing. The process goes on until the last element.

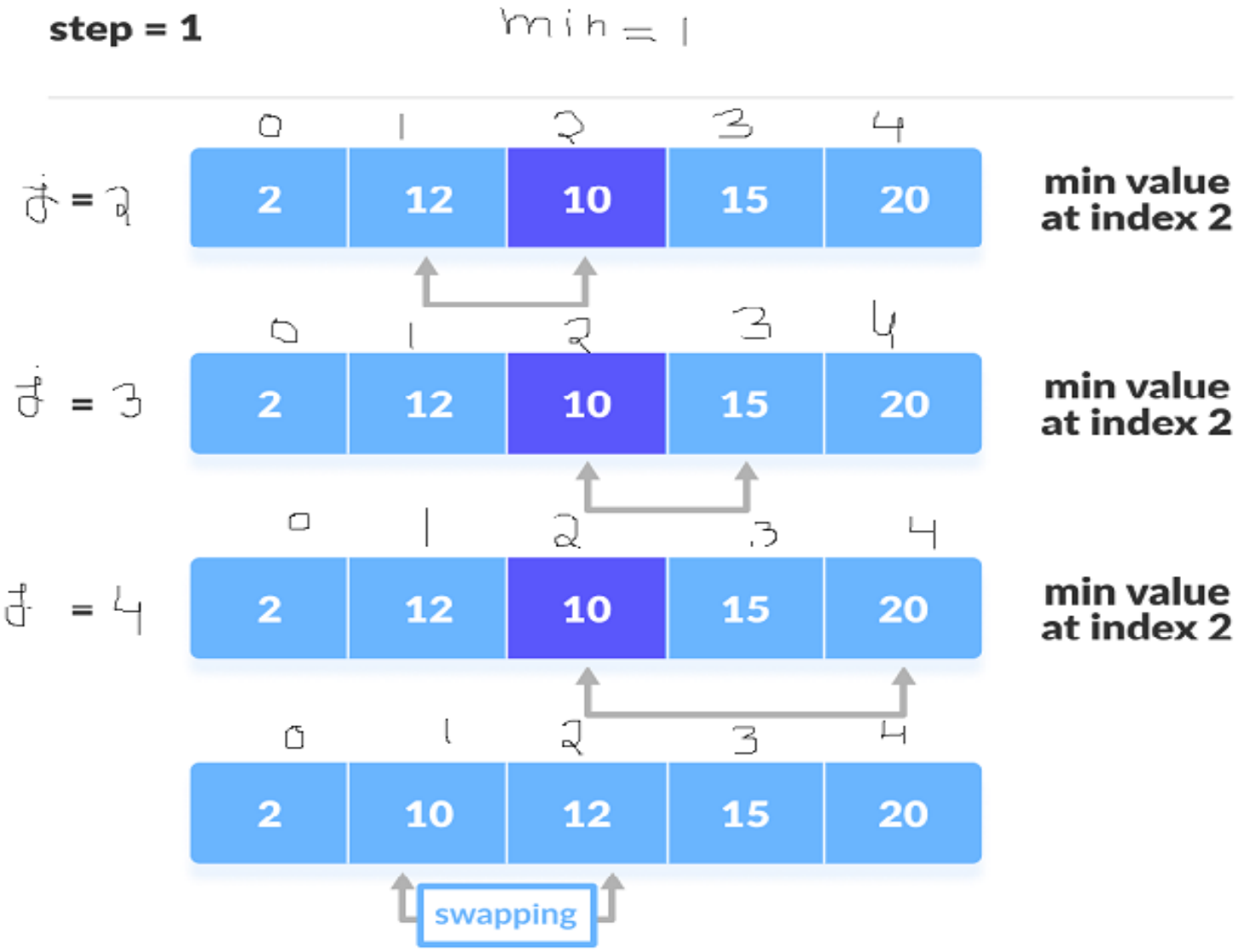
4. After each iteration, minimum is placed in the front of the unsorted list

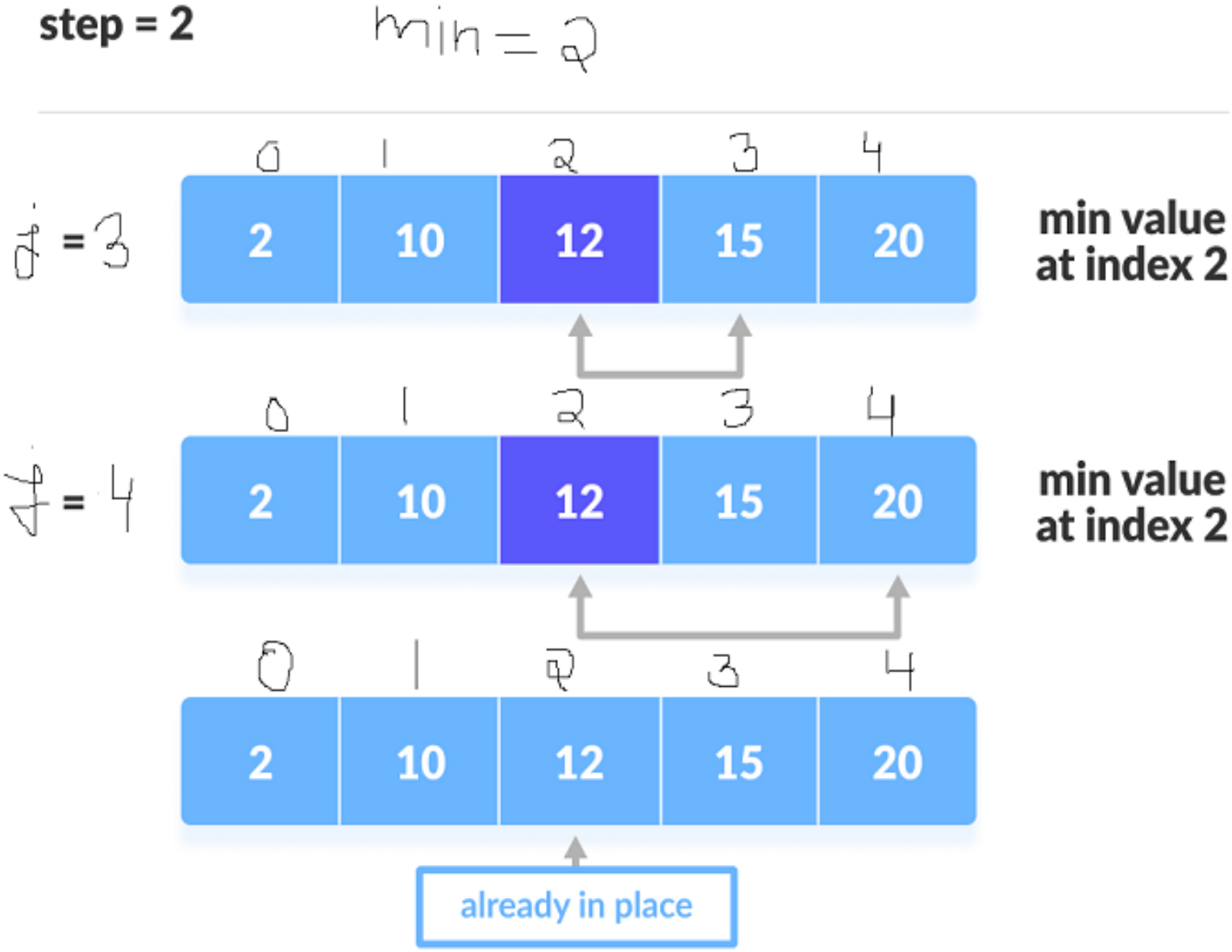


5. For each iteration, indexing starts from the first unsorted element. Step 1 to 4 are repeated until all the elements are placed at their correct positions.

Example of Selection Sort

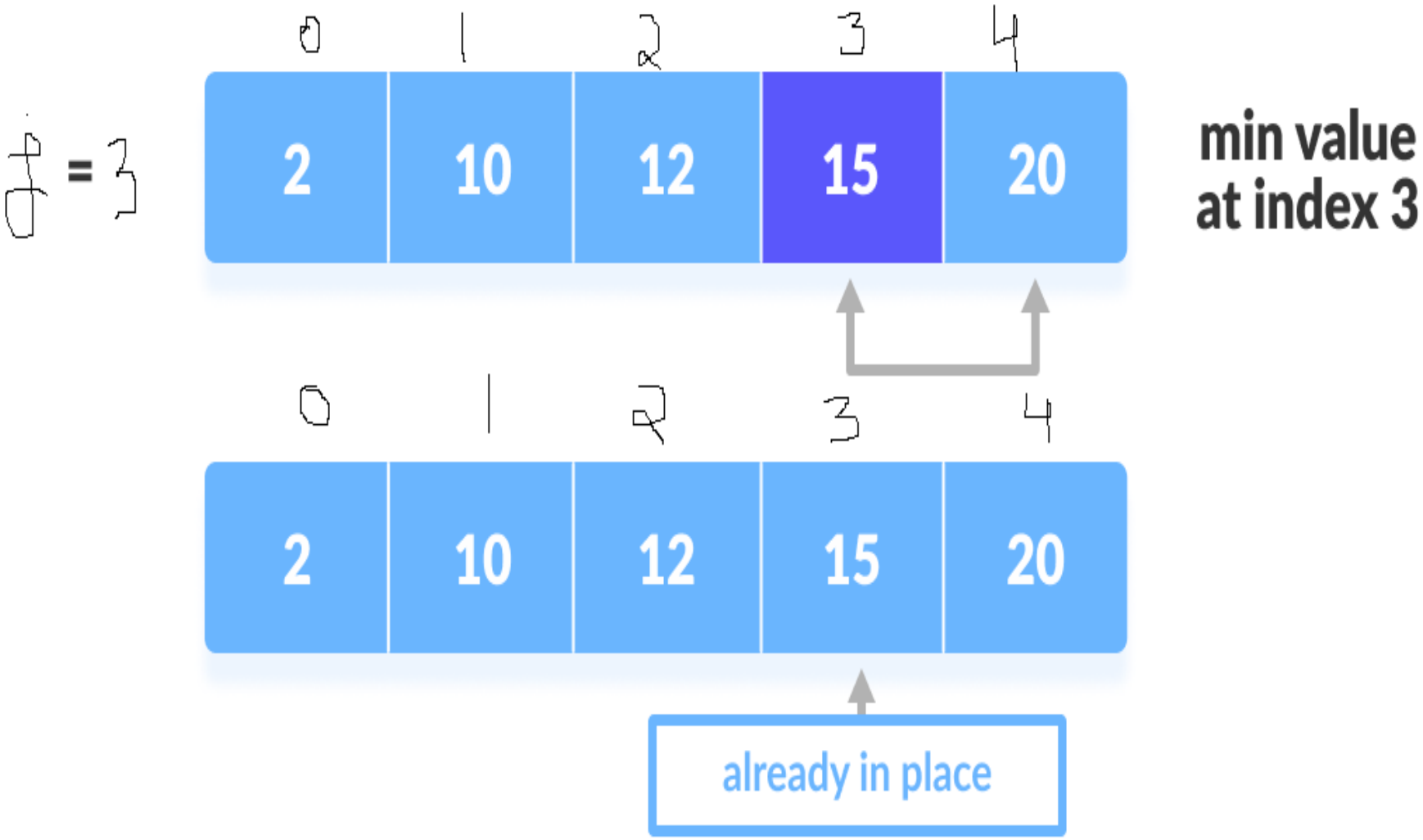

step = 0
min = 0
0 1 2 3 4
20 12 10 15 2
min value at index 1
i = 1
0 1 2 3 4
20 12 10 15 2
min value at index 2
i = 2
0 1 2 3 4
20 12 10 15 2
min value at index 2
i = 3
0 1 2 3 4
20 12 10 15 2
min value at index 4
i = 4
0 1 2 3 4
2 12 10 15 20
swapping


step = 1
min = 1
0 1 2 3 4
2 12 10 15 20
min value at index 2
0 1 2 3 4
2 12 10 15 20
min value at index 2
0 1 2 3 4
2 12 10 15 20
min value at index 2
0 1 2 3 4
2 10 12 15 20
swapping


step = 2
min = 2
0 1 2 3 4
2 10 12 15 20
min value at index 2
0 1 2 3 4
2 10 12 15 20
min value at index 2
0 1 2 3 4
2 10 12 15 20
already in place

step = 3

min = 3


0
1
2
3
4
2
10
12
15
20
min value at index 3
0
1
2
3
4
2
10
12
15
20
already in place

step = 3


i = 0
2
10
12
15
20
min value
at index 3
2
10
12
15
20
already in place

Algorithm of Selection Sort

Selectionsort(a[n])

Here a is an array of size n and this algorithm sort an array a.

```
Step 1: Start
Step 2:  for(i=0; i<=n-2; i++)
                    {
                        min = i;
                        for( j = i+1; j <= n-1; j++)
                            {
                                if( a[min] > a[j] )
                                    {
                                        min = j;
                                    }
                            }
                          if( min != i )
                            {
                                temp = a[i];
                                a[i] = a[min];
                                a[min] = temp ;
                            }
                    }
Step 3: Stop
```

Complexity of Selection Sort

Pass	Number of Comparison
1st	(n-1)
2nd	(n-2)
3rd	(n-3)
...	...
n-1	1

Total Number of comparisons will be

sum = (n-1) + (n-2) + (n-3) +.....+ 1

sum = n(n-1)/2

sum= $(n^2-n)/2$

i.e $O(n^2)$

Complexity (time Complexity) of selection sort = $O(n^2)$